Helwan Linux

Mastering PKGBUILD

From Beginner to Pro

مقدمة: فن التغليف فيArch Linux

من الاستهلاك إلى الصناعة

في عالم لينكس، هناك نوعان من المستخدمين: مستخدم يكتفي بما تقدمه له التوزيعة، ومستخدم يرفض أن يكون مجرد متفرج، فيقرر أن يفكك النظام ويعيد بناءه ليتناسب مع احتياجاته الخاصة. إذا كنت ممسكاً بهذا الكتاب، فأنت بالتأكيد من النوع الثاني.

ليست **Arch Linux**مجرد توزيعة؛ إنها فلسفة تقوم على البساطة (KISS) والتحكم الكامل. وفي قلب هذه الفلسفة يقع نظام بناء الحزم .(**Arch Build System**) تعلم كيفية كتابة ملف **PKGBUILD**ليس مجرد مهارة تقنية لترتيب الملفات، بل هو "امتلاك لمفاتيح النظام". هو الفرق بين أن تنتظر مطوراً ليتكرم بتحديث برنامجك المفضل، وبين أن تقوم أنت ببنائه من السورس كود في ثوانٍ معدودة.

لماذا هذا الكتاب؟

لقد صُمم هذا الكتاب ليكون دليلاً عملياً، يتدرج بك من الصفر المطلق حتى الاحتراف. لن نكتفي بشرح الأوامر، بل سنغوص في "المنطق" الذي يحرك كل سطر برمجي. ستتعلم كيف تحترم نظامك، وكيف تبني حزماً نظيفة تتبع معايير مجتمع Arch العالمي.

ما ستكتشفه في هذه الرحلة:

- **بناء الأساس :**فهم فلسفة البناء والتعامل مع سكريبتات.Bash
- **دورة حياة الحزمة :**من لحظة تحميل السورس كود وحتى استقراره في مجلدات النظام.
- **إدارة العلاقات :**كيف تجعل الحزم تتعايش وتتكامل عبر Dependencies و.Conflicts
- **الاحتراف المتقدم :**التعامل معGit ، وتقسيم الحزم الضخمة.(Split Packages)
- **فن التنظيف :**اكتشاف الأخطاء (Debugging) قبل أن تصل لنظامك.

لمن هذا الكتاب؟

سواء كنت طالباً يخطو خطواته الأولى في عالم "ديف أوبس(DevOps) "، أو مطوراً يريد نشر برامجه على مستودع AUR، أو حتى هاوٍ يريد فهم ما يدور خلف الكواليس؛ هذا الكتاب هو رفيقك.

تذكر دائماً: فيArch Linux ، **أنت القائد** .وهذا الكتاب هو خريطة الطريق الخاصة بك لتصبح "مُغلف حزم (Packager) " محترف.

لنبدأ الرحلة.. السطر الأول يبدأ من هنا.

المرحلة صفر : ا لأساس اللي لازم يكون موجود) لو ناقصك جزء هنا، كمّله الأول(

==========

0-1فهم فلسفةArch

إيه هيArch Linux ؟

KISS principle

ليه مفيش auto build system زيDebian ؟

0-2أساسيات) Bash ضروري جدًا (لازم تكون فاهم:

variables

arrays

if / case

functions

exit codes

quoting & escaping

PKGBUILD = Bash Script

اللي مش فاهم Bash عمره ما يحترفPKGBUILD

==========

إيه هيArch Linux ؟

: Arch Linuxهي توزيعة لينكس مستقلة بتركّز على البساطة، التحكم الكامل، وأحدث البرمجيات

.(rolling release) بدأت سنة 2002 و معنى نظام) Rolling Release مفيش إصدارات كبيرة كل سنة —

التحديثات مستمرة (موجهة للمستخدمين اللي عايزين يفهموا النظام بعمق , التثبيت بيكون يدوي

) من غير واجهة رسومية افتراضيًا (و أشهر حاجة فيها) Arch Wiki : مرجع قوي جدًا(

KISS Principle ✨في! Arch

KISS = Keep It Simple, Stupid

, لكن "Simple" هنا معناها : بسيط في التصميم ...مش بالضرورة سهل للمبتدئين .و السبب هنعرفه مع الفقرة القادمة

في:Arch

-مفيش أدوات تخفي عنك التفاصيل

-مفيش إعدادات افتراضية كتير

-مفيش تعديلات تقيلة على الباكدجات

-كل حاجة شبه ما المطور الأصلي نزلها

يعني النظام" نضيف "جدًا وقريب من.upstream

🤔ليه مفيش Auto Build System زيDebian ؟

خلينا نقارن بينها وبين:Debian

🌑 Debian

-عنده بنية ضخمة

-نظام Build farms رسمي

-سياسات صارمة جدًا

-بيركز على الاستقرار قبل أي حاجة

-الباكدجات بتتعدل أحيانًا عشان تناسب فلسفةDebian

● **Arch Linux**

-فلسفته مختلفة تمامًا

-بيستخدم نظام بسيط اسمهPKGBUILD

-أي حد يقدر يبني باكدج بنفسه باستخدامmakepkg

-مفيش طبقات معقدة من السياسات

-بيعتمد على صيانة أخف وأسرع

Archبيهدف إلى:

السرعة + البساطة + تقليل التعقيد الإداري

لو عملوا build system معقد زي:Debian

-هيزود البيروقراطية

-هيبطأ التحديثات

-هيكسر مبدأKISS

🎁طب بيبنوا الباكدجات إزاي؟

==========

المطور يكتب ملف PKGBUILD و يتم بناء الباكدج في بيئة نظيفة ثم يتم رفعه لل , repositories المستخدم

يثبت باستخدام pacman بالإضافة لكده فيه:

- AUR (Arch User Repository)

-مجتمع ضخم بيشاركPKGBUILD

-تقدر تبني أي حاجة تقريبًا بنفسك

) Bash Basics (مختصر جداً)

1 Variables (المتغيرات)

```
name="Ali"
echo $name
```

⚠ مفيش مسافات حوالين=

✔ الصح x=5 :

✘ غلط x = 5 :لو القيمة فيها مسافات:

```
msg="Hello World"
```

2 Arrays (المصفوفات)

```
arr=("one" "two" "three")
echo ${arr[0]}    # one
echo ${arr[@]}    # العناصر كل
echo ${#arr[@]}   # العناصر عدد
```

في PKGBUILD هتشوف شكل زي:

```
depends =("glibc" "gcc" "python")
```

3 if

```
if [[ $x -gt 5 ]]; then
   echo "كبير"
else
   echo "صغير"
fi
```

مقارنات مهمة:

= eq-يساوي

= ne-لا يساوي

= gt-أكبر من

= lt-أصغر من

4 case

بديل أنضف من if كتير:

```
case $1 in
  start)
    echo "تشغيل"
    ;;
  stop)
    echo "إيقاف"
    ;;
  *)
    echo "معروف مش"
    ;;
esac
```

5 Functions (الدوال)

```
say_hi() {
  echo "Hello $1"
}
```

say_hi Ali

في PKGBUILD هتشوف دايمًا:

```
build() {
   make
}
package() {
   make install
}
```

6 Exit Codes (جدًا مهم)

كل أمر بيرجع رقم:

= 0نجاح✅

أي رقم غير = 0 خطأ❌

مثال:

```
if make; then
   echo "تمام"
else
   echo "في مشكلة"
fi
```

تقدر تستخدم:

```
exit 1
```

عشان توقف السكريبت.

7 Quoting & Escaping (نقطة أخطر)

◆ Double quotes " "

بتسمح بتوسيع المتغير:

```
name="Ali"
echo "Hello $name"
```

◆ Single quotes ' '

بتمنع التوسيع:

```
echo 'Hello $name'
```

هيطبع $name حرفيًا.

◆ Escaping

```
echo "He said \"Hi\""
```

✿ليه ده مهم لPKGBUILD ؟

لأن100% PKGBUILD = Bash Script :

يعني:

المتغيرات = معلومات الباكدج(pkgname, pkgver)

arrays = dependencies

functions = build() / package()

exit codes = البناء فشل

quoting = يمنع bugs خطيرة

==========

●المرحلة — 1 : أول PKGBUILD في حياتك) بأبسط شكل ممكن(

الهدف من المرحلة دي مش" نحترف "ولا " نفهم كل حاجة"

🎯الهدف الوحيد:

تعملPKGBUILD

وتشغّلmakepkg

ويطلع باكدج شغال

وتفهم كل سطر ليه موجود

1.1هنشتغل على إيه؟

مش برنامج

مشC

مشPython

👉ملف نصي واحد يتثبت فيusr/bin/

يعني:

مفيشbuild

مفيشcompile

مفيش وجع دماغ

1.2حضّر المكان) افهم قبل ما تكتب (اعمل فولدر فاضي:

mkdir hello-pkg

cd hello-pkg

الفولدر ده هيبقى:

مكان البناء

مكان التجربة

مفيهوش أي خطر على النظام

1.3الملف اللي هنعمله باكدج جوه الفولدر اعمل ملف اسمه:hello

```
nano hello
```

محتواه:

```
#!/bin/bash
echo "Hello from my first PKGBUILD"
```

وبعدين:

```
chmod +x hello
```

📌**لحد هنا :ده برنامج Bash بيطبع جملة يشتغل لو كتبناhello/.**

1.4دلوقتي بقى :ملف PKGBUILD نفسه , أنشئ ملف اسمه) PKGBUILD بالضبط كده، حروف كبيرة:(

```
nano PKGBUILD
```

وحط فيه:

```
pkgname=hello-pkg
pkgver=1.0
pkgrel=1
pkgdesc="My first simple package"
arch=('any')
license=('MIT')
source=('hello')
sha256sums=('SKIP')
package() {
```

```
  install -Dm755 hello "$pkgdir/usr/bin/hello"
}
```

اقفل هنا ✋ مش هنكمّل غير لما نفهم كل سطر.

1.5نفهم السطور واحدة واحدة) على مهلك(

pkgname

pkgname=hello-pkg

اسم الباكدج ده اللي pacman هيشوفه لازم يكون:

حروف صغيرة

بدون مسافات

pkgver

pkgver=1.0

إصدار البرنامج , أي رقم لان توزيعة Arch مش بيفرض شكل معين

pkgrel

pkgrel=1

رقم إعادة التغليف , لو عدّلت PKGBUILD من غير ما تغيّر البرنامج → تزود ده

pkgdesc

pkgdesc="My first simple package"

وصف للإنسان مش للآلة

arch

arch=('any')

يعني :مش مرتبط بمعالج , مجرد ملف نصي

license

license=('MIT')

مش هنناقش الترخيص دلوقتي , المهم يكون موجود

source

source=('hello')

بتقول لـ :makepkg السورس عندي محليًا

sha256sums

sha256sums=('SKIP')

بنقول :متتحققش

ليه؟ لأننا لسه مبتدئين

وهنتعلم الصح بعدين

1.6أهم جزء في الملف كله()package :

```
package() {
  install -Dm755 hello "$pkgdir/usr/bin/hello"
}
```

دي معناها حرفيًا :خد ملف hello انسخه إلى /usr/bin/hello :بس جوه الباكدج مش النظام الحقيقي

وهمي مجلد = $pkgdir 📌

Archبيبني فيه الباكدج بأمان

]ملاحظة[

الـ D- بتعمل المسار، و m755- بتخلي الملف قابل للتشغيل؛

1.7اللحظة الحاسمة🔥

نفّذ makepkg :لو كل حاجة و لو صح :هيطلع ملفpkg.tar.zst.

ثبّته:

```
sudo pacman -U hello-pkg-1.0-1-any.pkg.tar.zst
```

وبعدين:

```
hello
```

🎉لو طلع النص اذاً كده أنت عملت أول باكدج في حياتك

1.8خلّي الحقيقة دي تثبت

أنت دلوقتي :ما حفظتش و ما نقلتش من AUR و كمان فاهم كل سطر وده أساس الاحتراف.

==========

🟢المرحلة — 2 فهم العمق الحقيقي Beginner) حقيقي(

هدف المرحلة

❌مش نزود تعقيد

❌مش نعمل برامج تقيلة

✅نفهم:

ليه $pkgdir مهم

ليه ممنوع نلمس النظام مباشرة

إزاي الباكدج بيتكوّن خطوة خطوة

2.1أهم مفهوم في PKGBUILD كله افهم الجملة دي كويس PKGBUILD :لا يثبت أي شيء على نظامك مباشرة

يعني:

مفيش سطر واحد بيكتب فيusr/

مفيش سطر واحد بيكسر النظام

2.2طب فين الملفات بتروح؟

لما تشغّل makepkg :مباشرة نظام التشغيل Arch بيعمل الآتي) خدها بالترتيب:(

ينشئ مجلد مؤقت

يسميه$pkgdir

يقولك:

"اعمل اللي انت عايزه ...بس جوا هنا"

يعني لما تكتب:

"$pkgdir/usr/bin/hello"

ده فعليًا بيبقى حاجة شبه:

/home/user/hello-pkg/pkg/hello-pkg/usr/bin/hello

📌مش /usr/bin/hello الحقيقي

2.3ليه ماينفعش ننسخ مباشرة لusr/ ؟) سؤال مهم جدًا(.

تخيل PKGBUILD فيهcp hello /usr/bin/hello :

ده معناه:

بيعدل على النظام أثناء البناء

لو فشل في النص → النظام اتلوث

pacmanمش هيعرف الملف ده جه منين

❌كارثة علشان كده:

Archقال :كل حاجة تروح pkgdir وبالتالي pacman هو الوحيد اللي يلمس النظام

2.4نفهم أمر install بعمق السطر ده:

```
install -Dm755 hello "$pkgdir/usr/bin/hello"
```

نفكّه حتة حتة.

install

مشinstall packages

ده أمر Unix : و ظيفته :نسخ + صلاحيات + إنشاء مجلدات

D-معناها :اعمل المجلدات لو مش موجودة

يعني usr/ :و كمان usr/bin/ هيتعملوا تلقائي

m755-الصلاحيات معناها:

owner:قراءة + كتابة + تنفيذ

group:قراءة + تنفيذ

others:قراءة + تنفيذ

مناسب للبرامج التنفيذية

2.5خلينا نزوّد حاجة صغيرة) تطبيق عملي (هنضيف ملف توثيق.

1 اعمل ملف:README

```
nano README
```

محتواه:

```
This is my first Arch package.
```

2 عدّل PKGBUILD خليه كده:

```
package() {
  install -Dm755 hello "$pkgdir/usr/bin/hello"
  install -Dm644 README "$pkgdir/usr/share/doc/hello-pkg/README"
}
```

نفهم السطر الجديد -m644 : ملف عادي , مش executable

المسار:

```
/usr/share/doc/hello-pkg/
```

ده المكان القياسي للتوثيق

3 ابني من جديد

```
makepkg -f
```

وبعدين:

```
sudo pacman -U hello-pkg-1.0-1-any.pkg.tar.zst
```

وشوف:

```
cat /usr/share/doc/hello-pkg/README
```

2.6 حقيقة مهمة جدًا) احفظها(

أي ملف مش جوه = $pkgdir مش هيبقى جزء من الباكدج

يعني:

نسيت تحطه → مش موجود

حطيته غلط → ضاع

مساره غلط pacman → مش هيشوفه

2.7أخطاء المبتدئين الشائعة) إوعى تقع فيها(

- ✕استخدام sudo داخلPKGBUILD
- ✕الكتابة في usr/ مباشرة
- ✕نسيان$pkgdir
- ✕يدوي بدلm- install chmod

ثبت اللي فات قبل ما نطلع لو فهمت دول:

$pkgdir

install

==========

المرحلة الثالثة source - و) checksums بالعقل مش بالحفظ(

هدف المرحلة تفهم:

- **يعني إيه source فعلًا**
- **ليه Arch متشدد فيchecksums**
- **إزاي تجيب سورس من:**
  - **ملف محلي**
  - **أرشيف**
  - **Git**

من غير ما تتلخبط

3.1يعني إيهsource ؟

خلّينا نبدأ بجملة واحدة source :هو كل ملف البرنامج محتاجه علشان يتبني

مش شرط يكون :كود لان برنامج كبير ممكن يكون :ملف واحد , أرشيف , سكربت او repo كامل

3.2اللي عملناه قبل كده كان إيه؟

في المرحلة اللي فاتت كتبنا:

source=('hello')

ده معناه:

السورس موجود في نفس الفولدر

makepkgمش هيحمّل حاجة

بس هيعمل copy داخلي

وده مناسب للتعلّم فقط مش للنشر.

3.3ليه checksums موجودة أصلًا؟

Archبيقولك:

أنا هابني من سورس

بس عايز أتأكد إن السورس ما اتلعبش فيه"

checksum = بصمة

لو الملف اتغير و لو حتى حرف واحد

→البصمة تختلف

→البناء يفشل

📌ده أمان، مش تعقيد.

3.4ليه استخدمنا SKIP قبل كده؟

علشان:

كنا بنتعلم

الملف محلي

مفيش تحميل

لكن:

❌مينفعش في باكدج حقيقي

❌مينفعش فيAUR

✕مينفعش في توزيعة

3.5نعمل مثال حقيقي) أرشيف(

1 حضّر سورس بسيط و اعمل فولدر جديد:

```
mkdir hello-src

cd hello-src
```

اعمل ملف:

```
nano hello.sh
```

محتواه:

```
#!/bin/bash

echo "Hello from archived source"
```

وبعدين:

```
chmod +x hello.sh
```

اعمله أرشيف:

```
tar -czf hello-1.0.tar.gz hello.sh
```

3.6 (PKGBUILD)بسورس حقيقي

ارجع لفولدر جديد للبناء:

```
cd ..

mkdir hello-archive-pkg

cd hello-archive-pkg
```

انسخ الأرشيف:

```
cp ../hello-src/hello-1.0.tar.gz .
```

اعمل PKGBUILD:

```
nano PKGBUILD
```

المحتوى:

```
pkgname=hello-archive
pkgver=1.0
pkgrel=1
pkgdesc="Hello package built from tar.gz"
arch=('any')
license=('MIT')
source=('hello-1.0.tar.gz')
sha256sums=('PUT_CHECKSUM_HERE')

package() {
  install -Dm755 hello.sh "$pkgdir/usr/bin/hello-archive"
}
```

3.7نجيب checksum الصح) مش يدوي(

نفّذ:

```
makepkg -g
```

هيطلع سطر زي:

```
sha256sums=('abc123...')
```

انسخه بالضبط

وحطه مكان.PUT_CHECKSUM_HERE

📌**دي الطريقة الصحيحة**

❌**عمرنا ما نحسبها بإيدينا**

3.8إيه اللي حصل جوهmakepkg ؟

لما شغّلت makepkg :مباشرة نظام Arch عمل الآتي:

فك الأرشيف

حط الملفات في$srcdir

نفّذ()package

خد الملفات من$pkgdir

عمل الباكدج

3.9يعني إيه$srcdir ؟

خلي الجملة دي في دماغك:

الفك بعد السورس مكان = $srcdir

يعني لو عندك:

hello.sh

هتلاقيه في:

$srcdir/hello.sh

📌وده سبب إننا نكتب في:()package

install -Dm755 hello.sh ...

مش من فولدر تاني.

3.10خطأ شائع جدًا

❌الكتابة كده:

install -Dm755 ../hello.sh ...

ده:

كسر

غير مضمون

يعتمد على مكانك

Archعايز:

$srcdir

$pkgdir

ولا حاجة غيرهم

3.11خلاصة المرحلة) مهمة(

لو فهمت دول:

source = السورس

sha256sums = الأمان

makepkg -g = الصح الطريقة

$srcdir ≠ $pkgdir

يبقى أنت عدّيت أول خط احترافي.

==========

المرحلة الرابعة Git sources - و) ()pkgver ببساطة شديدة(

هدف المرحلة تفهم:

يعني إيه باكدجgit-

إزاي makepkg يتعامل معGit

ليه ()pkgver مهمة

تعمل باكدج Git شغال من غير لخبطه

4.1يعني إيه باكدجgit- ؟

خلّينا نبدأ بجملة واحدة:

باكدج git- = باكدج بيتبني من أحدث كود موجود في Git

يعني:

مفيش إصدار ثابت(2.3 / 1.0)

كل build ممكن يكون مختلف

مناسب للتطوير، مش للاستقرار

علشان كده:

اسمه دايمًا بينتهي بـgit-

ماينفعش يبقى نفس اسم النسخة الثابتة

مثال:

foo → نسخة مستقرة

foo-git → نسخة Git

4.2نعمل مثال عملي) بسيط جدًا (مش هنستخدم مشروع تقيل هنعمل repo وهمي صغير علشان الفهم.

1 اعمل repo Git بسيط

```
mkdir hello-git-src

cd hello-git-src

git init
```

اعمل ملف:

```
nano hello.sh
```

محتواه:

```
#!/bin/bash

echo "Hello from git package"
```

وبعدين:

```
chmod +x hello.sh

git add hello.sh

git commit -m "first commit"
```

4.3 PKGBUILD لباكدج Git

اخرج لفولدر تاني:

```
cd ..

mkdir hello-git-pkg

cd hello-git-pkg
```

اعمل:PKGBUILD

```
nano PKGBUILD
```

المحتوى:

```
pkgname=hello-git

pkgver=0

pkgrel=1

pkgdesc="Hello package built from git source"

arch=('any')

license=('MIT')

depends=('bash')

source=("git+file:///$(pwd)/../hello-git-src")

sha256sums=('SKIP')

pkgver() {

  cd "$srcdir/hello-git-src"

  git describe --long --tags 2>/dev/null || echo "r$(git rev-list --count HEAD).$(git rev-parse --short
HEAD)"

}

package() {

  cd "$srcdir/hello-git-src"

  install -Dm755 hello.sh "$pkgdir/usr/bin/hello-git"

}
```

اقفل هنا ✋ دلوقتي نفهم كل جزء بهدوء.

4.4نفهم source معGit

```
source=("git+file:///...")
```

ده معناه:

المصدرGit repo

makepkgهيعمل:

clone

update

checkout

📌أي Git URL ينفع:

local

https

ssh

4.5ليه ('SKIP')=sha256sums هنا مسموح؟ سؤال ذكي🧠

علشان:

Gitبيتغير

مفيش ملف ثابت نعمل لهchecksum

Archيسمح بـ SKIP لـ Git فقط

❌ينفعش مع tar.gz حقيقي

❌ينفعش مع ملفات ثابتة

4.6يعني إيه()pkgver ؟

خد الجملة دي كويس:

()pkgverبتحدد رقم الإصدار وقت البناء

يعني:

makepkgيناديها

ترجع نص

النص ده يتحط مكانpkgver

السطر ده:

git describe --long --tags

لو فيه → tags يستخدمها

لو مفيش:fallback →

r<commit-count>.<short-hash>

مثال:

r3.a1b2c3

وده :رقم منطقي بيتغير مع كلcommit

pacmanيقدر يقارن بين الإصدارات

4.7ليه ما نسيبش pkgver ثابت؟

لو عملتpkgver=1.0 :

في باكدج pacman :Git مش هيعرف إن فيه تحديث

صح هيشتغل مش Syu-

كده كسرت الفلسفة

4.8 build عملي

نفّذ:

```
makepkg
```

وبعدين:

```
sudo pacman -U hello-git-*.pkg.tar.zst
```

وشغّل:

```
hello-git
```

لو طلع النص → تمام

4.9 أخطاء قاتلة في باكدجات Git

- ❌ **نسيان pkgver()**
- ❌ **استخدام checksum مع Git**
- ❌ **تسمية الباكدج بدون -git**
- ❌ **الاعتماد على مسارات خارج $srcdir**

4.10 ثبت اللي فات

لو فاهم:

ليه -git

```
source git+...
pkgver()
```

الفرق بين ثابت و Git

يبقى أنت دلوقتي:

- **Beginner → Intermediate**

==========

المرحلة الخامسة) dependencies - الفرق اللي بيعمل محترف(

هدف المرحلة تفهم بوضوح:

يعني إيه dependency أصلًا

الفرق بين:

depends

makedepends

optdepends

إمتى تحط الحاجة هنا أو هنا

وإزاي ما تكسّرش نظام المستخدم

5.1يعني إيهdependency ؟

خلّينا نبدأ من الصفر:

dependency = حاجة برنامجك محتاجها علشان يشتغل أو يتبني

مش رفاهية

مش اقتراح

حاجة ضرورية

5.2أول نوعdepends :

```
depends=('bash')
```

معناها:

البرنامج مش هيشتغل من غيرها pacman يثبتها تلقائي

مثال بسيط :لو عندك سكربتBash:

محتاج bash وقت التشغيل

→تحطها فيdepends

❌غلطة شائعة :تحط حاجة في depends وهي مش مطلوبة وقت التشغيل

5.3النوع التانيmakedepends :

makedepends=('gcc' 'make')

معناها:

مطلوبة وقت البناء فقط

المستخدم النهائي مش محتاجها

مثال:

برنامج :C محتاج gcc علشان يتبني لكن بعد ما يتبني :الملف التنفيذي شغال لوحده

📌إذن:

gcc → makedepends

مشdepends

5.4الفرق العملي) مهم جدًا(

الحاجة	تحطها فين؟
مطلوبة وقت التشغيل	depends
مطلوبة وقت البناء فقط	makedepends
اختيارية	optdepends

5.5النوع التالتoptdepends :

optdepends=('vim: edit configuration')

معناها:

البرنامج يشتغل من غيرها , لكن التجربة أحسن بيها

مثال :برنامج CLI يفتح editor لو vim موجود → تجربة أحسن لو مش موجود → عادي

pacman 📌يعرضها للمستخدم لكن مش يفرضها

5.6نعدّل مثالنا السابق) تطبيق عملي(

خلّينا نعدّل باكدج .hello-git في:PKGBUILD

```
depends=('bash')

optdepends=('figlet: fancy output')
```

ليه؟

bashضروري

figletمجرد شكل حلو

5.7غلطة قاتلة شائعة

❌تحط كل حاجة فيdepends

❌تخلي المستخدم يثبت حاجات مش محتاجها

❌تخلي النظام تقيل بلا سبب

Archيحب:

أقل عددdependencies

أكبر تحكم للمستخدم

5.8 dependency مش معناهاcommand

دي نقطة ناس كتير بتغلط فيها.

❌غلط:

```
depends=('ls')
```

✅صح:

coreutils من جزء ls

تحط coreutils لو فعلاً محتاجه) نادر(

5.9إزاي أعرف أحط الحاجة فين؟

اسأل نفسك السؤال ده:

لو شلت الحاجة دي بعد تثبيت الباكدج هل البرنامج يشتغل؟

لا → depends

آه → makedepends أو optdepends

5.10تمرين عقلي) من غير كتابة(

عندك برنامج :مكتوب Python بيتبني بـ setuptools بيطلع script يشتغل بـpython

فين تحط:

python → ؟

setuptools → ؟

الإجابة:

python → depends

setuptools → makedepends

لو جاوبت كده تلقائي → أنت فهمت.

5.11خلاصة المرحلة

أنت دلوقتي فاهم:

dependencyمش عشوائي

Archصارم علشان النظام يفضل نظيف

اختيارك للمكان = احترامك للمستخدم

وده فرق محترف عن ناس كتير في.AUR

==========

🔘المرحلة السادسة) - prepare() / build() / check() - ليه فيه أكتر منfunction ؟(

هدف المرحلة تفهم:

كل function دورها إيه

إمتى تستخدمها

الترتيب الحقيقي للتنفيذ

وإزاي تكتب PKGBUILD نظيف يفهمه أي حد بعدك

6.1الترتيب الحقيقي اللي Arch بينفّذه

خد الترتيب ده واحفظه كمنطق مش حفظ أسماء:

1 prepare()

2 build()

3 check()

4 package()

!ولا واحدة منهم إجبارية

!لكن كل واحدة ليها وظيفة واضحة

— 6.2 prepare()التحضير قبل أي حاجة

معناها ببساطة" :حضّر السورس قبل البناء"

تستخدمها لما :تعدّل ملفات

تطبّقpatch

تغيّرpermissions

تجهّزconfig

مثال بسيط:

```
prepare(){
  cd "$srcdir/project"
  sed -i 's/DEBUG=1/DEBUG=0/' config.mk
}
```

📌هنا :ما بنبنيش بس بنعدّل السورس

❌غلطة شائعة :تحط build فيprepare

— 6.3 build()البناء الحقيقي

معناها :حوّل السورس لبرنامج جاهز”

هنا بتحط:

```
make
cmake
meson
python setup
```

أي compile

مثال بسيط:

```
build() {
  cd "$srcdir/project"
  make
}
```

📌 build: لا يكتب في النظام , لا ينسخ ملفات نهائية و ظيفته بس يبني

— 6.4 check() (الاختبارات) اختياري

معناها:

"تأكّد إن اللي اتبنى شغال صح"

تستخدمها لو:المشروع فيه

```
tests
make test
pytest
ctest
```

مثال:

```
check() {
  cd "$srcdir/project"
  make test
}
```

📌لو tests فشلت:

makepkg يفشل

ده شيء كويس مش وحش

— 6.5 package() (التغليف النهائي) الأهم

معناها:

انسخ الناتج النهائي إلى $pkgdir

هنا فقط:

```
install
```

copy

permissions

docs

configs

❌ممنوع:

build

patch

download

6.6مثال كامل) صغير وواضح(

خلّينا نشوف PKGBUILD منظم:

```
pkgname=hello-structured
pkgver=1.0
pkgrel=1
pkgdesc="Structured example"
arch=('any')
license=('MIT')
depends=('bash')
source=('hello.sh')
sha256sums=('SKIP')
prepare() {
  chmod +x hello.sh
}
```

```
build() {
  echo "Nothing to build"
}
package() {
  install -Dm755 hello.sh "$pkgdir/usr/bin/hello-structured"
}
```

📌هنا:

prepare → تجهيز

build → مفيش build (جدًا عادي)

package → التغليف

6.7غلطة شائعة جدًا

❌تحط كل حاجة في()package

ليه غلط؟

صعب تقرأ

صعب تصحّح

يخالف فلسفةArch

Archيحب :كل مرحلة في مكانها

6.8إمتى أسيب function فاضية؟

عادي جدًا:

لو مفيش → prepare شيلها

لو مفيش → build شيلها

لو مفيش → check شيلها

❌ما تسيبهاش فاضية بدون سبب

6.9تمرين ذهني) مهم(

لو عندك برنامج :محتاج patch وبعدها compile وبعدهاinstall

الترتيب الصح:

patch → prepare

compile → build

install → package

لو جاوبت كده تلقائي → أنت تمام.

6.10خلاصة المرحلة

دلوقتي أنت:

فاهم lifecycle الباكدج

تقدر تقرأ PKGBUILD لأي مشروع

تعرف السطر مكانه صح ولا غلط

==========

🟤المرحلة السابعة) - Split Packages - باكدج واحد → أكتر من باكدج(

هدف المرحلة تفهم:

يعني إيهsplit package

ليه Arch بيستخدمه

إزاي تكتبه صح

وإزاي ما تلخبطش pacman ولا المستخدم

7.1يعني إيهSplit Package ؟

خلّينا نبدأ بالجملة الذهبية = Split package :سورس واحد نطلع منه أكتر من باكدج

يعني:

buildواحد

PKGBUILDواحد

لكن الناتج :باكدج - CLI باكدج - GUI باكدج - docs باكدجlibs

7.2ليه نعمل كده؟

علشان:

المستخدم يثبت اللي محتاجه بس

النظام يفضل خفيف

dependenciesتقل

مثال واقعي :برنامج فيه:

مكتبة

أداة سطر أوامر

واجهة رسومية

مش منطقي :تثبّت GUI على سيرفر

7.3الشكل الأساسي لsplit package
بدلpkgname=foo :

نكتب:

pkgname=(foo foo-cli foo-docs)

📌كل اسم هنا :باكدج مستقل و ليه files خاصة و كمان ليه dependencies مختلفة

7.4 structureمهم جدًا فيsplit package:

pkgbase = اسم المشروع

pkgname = أسماء الباكدجات

مثال:

```
pkgbase=hello
pkgname=(hello-cli hello-docs)
```

7.5 package_*() — القلب الحقيقي

كل باكدج ليه function خاصة:

```
package_hello-cli() {
  install -Dm755 hello.sh "$pkgdir/usr/bin/hello"
}
package_hello-docs() {
  install -Dm644 README "$pkgdir/usr/share/doc/hello/README"
}
```

📌لاحظ:

$pkgdir بيتغير لكل باكدج تلقائي

ما تعملش مجلدات يدوي

7.6 مثال كامل (واضح جدًا)

```
pkgbase=hello
pkgname=(hello-cli hello-docs)
pkgver=1.0
pkgrel=1
pkgdesc="Hello split package example"
arch=('any')
license=('MIT')
depends=('bash')
```

```
source=('hello.sh' 'README')
sha256sums=('SKIP' 'SKIP')
package_hello-cli() {
  install -Dm755 hello.sh "$pkgdir/usr/bin/hello"
}
package_hello-docs() {
  install -Dm644 README "$pkgdir/usr/share/doc/hello/README"
}
```

النتيجة:

hello-cli.pkg.tar.zst

hello-docs.pkg.tar.zst

7.7 dependencies في split packages

دي نقطة ناس كتير بتغلط فيها.

❌غلط:

```
depends=('bash')
```

ويطلع الباكدجين يعتمدوا عليه

✅صح:

كل باكدج ليه depends الخاصة بيه

مثال:

```
depends=('bash')
package_hello-docs() {
  depends=()
  install -Dm644 README "$pkgdir/usr/share/doc/hello/README"
}
```

📌 docs غالبًا :مفيهاشdependencies

7.8غلطة قاتلة

❌تحط كل الملفات في ()package واحدة

❌تنسى تقسّمها صح

❌تخلي docs تجرGUI

split packageهدفه:

تحكّم

مش تعقيد

7.9إمتى تستخدمsplit packages ؟

استخدمها لو :المشروع كبير فيه أكتر من component او المستخدم مش محتاج كله

❌ما تستخدمهاش :علشان" شكلها احترافي "او لو المشروع صغير

7.10تمرين ذهني

برنامج فيه:

binary

man page

README

تقسّمه إزاي؟

الإجابة:

foo → binary

foo-docs → README

foo-man → man page (اختياري)

لو وصلت للإجابة دي لوحدك → ممتاز.

7.11خلاصة المرحلة

دلوقتي أنت:

فاهمsplit packages

تعرف تقرأ PKGBUILD تقيل

مش هتتلخبط لما تشوف()=pkgname

وده مستوى ناس كتير ما بتوصلهوش.

==========

🟤المرحلة — 8 ملفات install. و) hooks إمتى نستخدمها وإمتى نعتبرها جريمة(

هدف المرحلة تفهم:

- **يعني إيه ملفinstall.**
- **إمتى نحتاجه فعلًا**
- **الفرق بينه وبينPKGBUILD**
- **أخطر الأخطاء اللي بتحصل بسببه**

8.1يعني إيه ملفinstall. ؟

خلّينا نبدأ بجملة واضحة :ملف = install. سكربت بيتنفّذ وقت التثبيت أو الإزالة و مش وقت البناء

ولا مش وقتmakepkg

📌وقت pacman فقط

8.2 PKGBUILD vs .install (فرق جوهري)

الحاجة	PKGBUILD	.install
وقت التنفيذ	وقت البناء	وقت التثبيت
يلمس النظام؟	❌لا	✅آه
يستخدمsudo ؟	❌لا	❌لا (pacman هو اللي(root

⚠مهم:

.installشغال كroot

أي غلطة = خطر

8.3إمتى نحتاج install. فعلًا؟

استخدمه فقط لو محتاج :

تحديث(cache (icons / desktop

reload systemd

رسالة للمستخدم

enable service (بحذر شديد)

❌ما تستخدموش علشان:

نسخ ملفات

build

تعديل config افتراضي

8.4شكل ملفinstall.

اسمه دايمًاpkgname.install :

مثال:

hello.install

8.5الدوال المتاحة جواه

post_install()

post_upgrade()

post_remove()

معناهم:

post_install()

→بعد التثبيت أول مرة

post_upgrade()

→بعد التحديث

post_remove()

→بعد الإزالة

8.6مثال بسيط وآمن

خلّينا نعمل ملف:

nano hello.install

محتواه:

```
post_install() {
  echo "Thank you for installing hello"
}
post_remove() {
  echo "Goodbye from hello"
}
```

📌ده :آمن مفيهوش تعديل على النظام و مجرد رسالة

8.7ربط install. بPKGBUILD

في PKGBUILD لازم تقول إن فيه:install file

```
install=hello.install
```

مثال:

```
pkgname=hello
pkgver=1.0
pkgrel=1
install=hello.install
```

من غير السطر ده :الملف يتجاهل ما يتنفذش

8.8مثال عملي مهم(systemd service)

سيناريو واقعي :برنامج بيضيفservice

❌غلط جدًا:

```
systemctl enable foo.service
```

ليه؟

لان المستخدم مش موافق

تدخل مباشر

ضد فلسفةArch

✅الصح:

```
post_install() {
```

```
  echo "You can enable the service with:"
  echo "  systemctl enable foo.service"
}
```

📌دايمًا:

أبلغ المستخدم — ما تفرضش

8.9غلطة قاتلة جدًا) إوعى تعملها(

❌حذف ملفات مش تابعة للباكدج فيpost_remove()

❌تعديل etc/ تلقائي

❌تشغيلnetwork commands

❌ enable services بدون إذن

لو شفت PKGBUILD بيعمل كده:

ده باكدج وحش

حتى لو شغال

8.10 hooks vs .install)فرق مهم(

.install:

خاص بالباكدج

بسيط

محدود

hooks:

system-wide

معقد

غالبًا مش للمبتدئ

📌كمبتدئ/متوسط:

ركّز علىinstall.

وسيب hooks دلوقتي

8.11خلاصة المرحلة

دلوقتي أنت فاهم:

install.خطر لو استُخدم غلط

PKGBUILDيبني، install. يثبّت

احترام المستخدم أهم من" السهولة"

==========

🌑المرحلة التاسعةconflicts / provides / replaces -

هدف المرحلة نفهم

ليه باكدج ممكن يتعارض مع باكدج تاني

إزاي نخلي pacman يعرف البدائل

نتعلم كتابة هذه الحقول بطريقة آمنة ومحترفة

9.1يعني إيهconflicts ؟

الجملة الذهبية:

conflicts = باكدجات لا يمكن تثبيتها مع الباكدج ده في نفس الوقت

مثال عملي:

```
pkgname=foo

conflicts=('foo-old')
```

المعنى:

إذا كان مستخدم مثبت foo-old قرار pacman يمنع التثبيت أو يقترح إزالة القديم أول

9.2إمتى تستخدمconflicts ؟

لما الباكدج يحتوي على نفس الملفات مثل باكدج تاني

لما النسخة الجديدة تستبدل القديمة بشكل كامل

❌خطأ شائع:

كتابة conflicts عشوائي

يمنع تثبيت باكدجات ما لهاش علاقة

9.3يعني إيهprovides ؟

الجملة الذهبية:

provides = هذا الباكدج يحل محل باكدج افتراضي أو واجهة

مثال:

```
pkgname=python-pypy

provides=('python')
```

المعنى:

أي برنامج محتاج python ممكن يشتغل مع python-pypy و هنا pacman يعترف بالحل البديل

9.4إمتى تستخدمprovides ؟

باكدج بديل لآخر

مشروع مختلف لكن يقدّم نفس الخدمة

مثال عملي:

foo-cli vs foo-gui

ممكن يكونوا يوفروا نفس المكتبة الخلفية

تقدر تحط provides=('foo-lib') للاثنين

9.5يعني إيهreplaces ؟

الجملة الذهبية:

replaces = هذا الباكدج سيحل محل باكدج محدد عند التثبيت

مثال:

```
pkgname=foo-new
replaces=('foo-old')
```

المعنى:

لما تثبّت foo-new هنا pacman يحذف foo-old تلقائي بدون خطأ أو تعارض

9.6مثال عملي كامل

```
pkgname=foo
pkgver=1.0
pkgrel=1
pkgdesc="Example of conflicts/provides/replaces"
arch=('any')
license=('MIT')
conflicts=('foo-old')
provides=('foo-lib')
replaces=('foo-old')
package() {
```

```
  install -Dm755 hello.sh "$pkgdir/usr/bin/hello"
}
```

النتيجة:

يحمي النظام من التعارضات

pacmanيعرف إيه يزيل وإيه يبقى

9.7غلطة قاتلة شائعة

❌تحط conflicts على باكدجات مش مترابطة → يمنع تثبيت المستخدم

❌تستخدم replaces بدون فهم → تحذف ملفات مهمة

❌تستخدم provides غلط pacman → يربكdependency resolution

9.8خلاصة المرحلة دلوقتي أنت تعرف:

→ conflictsيمنع التثبيت مع القديم

→ providesيعطي" واجهة بديلة"

→ replacesيستبدل القديم مباشرة

==========

🔘المرحلة العاشرةDebugging & Common Mistakes -

هدف المرحلة

تعرف إزاي تكتشف سبب فشلmakepkg

تتجنب الأخطاء اللي معظم المبتدئين بيوقعوا فيها

تعمل باكدج تعمل من أول مرة أو على الأقل تعرف تصلحه بسرعة

10.1أهم خطوة :قراءة الـ output كويس لما تعمل:

makepkg

خطواتك :اقرأ الرسائل اللي ظهرت بالترتيب دور على كلمات زي:

error

failed

cannot find

ركّز على الملف أو السطر اللي عمل المشكلة

نصيحة :لا تهمل أيWarning ، أحيانًا التحذيرات تكشف مشاكل مستقبلية

10.2 common mistakes في PKGBUILD

الغلطة	السبب	الحل
نسيت $pkgdir	ملفات تروح النظام استعمل	install -D و$pkgdir
استخدمت sudo	تغيير النظام أثناء البناء	لا تستخدم sudo — pacman
نسيان checksum	security fail	استخدم makepkg -g للحصول على sha256
تحط كل شيء في package()	اختلاط التحضير والبناء	استخدم prepare(), build(), package() صح
نسيت dependencies	البرنامج يفشل بعد التثبيت	حدد depends و makedepends بذكاء
تحط ملفات خارج $srcdir	makepkg يفشل	انسخ الملفات أولًا في $srcdir
replace / conflicts غلط	pacman يحذف أو يمنع باكدجات	تأكد من صحة العلاقات
install.يقوم بعمل root modification خاطئ	ممكن يكسر النظام استخدمه فقط لرسائل، أو system hooks بطريقة آمنة	

10.3 كيف تعالج فشل checksum

لو تستخدم Git → ضع 'SKIP'

لو تستخدم tar.gz → لا تضع 'SKIP'

شغل:

makepkg -g

انسخ البصمة الناتجة إلى sha256sums

نصيحة checksum :يحمي المستخدم من ملفات فاسدة أو تم التلاعب فيها

10.4 debugللأوامر المفقودة

لو makepkg يقول:

command not found

حلها :أضف البرنامج إلى makedepends أو تأكد إنه مثبت على النظام

10.5نصائح عملية لتقليل الأخطاء

استخدم PKGBUILD صغير وبسيط في البداية

اختبر كل:stage

prepare()

build()

check()

package()

استخدم makepkg -f لتجربة إعادة البناء بسرعة

لا تسرع في split packages أو install. حتى تتقن الأساسيات

اقرأPKGBUILD ات موجودة في AUR المشهورة كمثال

10.6طريقة محترف للتحقق

makepkg -s --nocheck

→ s-يثبت dependencies تلقائي

→ nocheck--يتجاوز ()check لو عايز تجربة سريعة

بعد نجاح → build شغّل ()check بعدين

هتقلل مشاكل أول تجربة بشكل كبير

10.7خلاصة المرحلة لو اتقنت:

قراءة الأخطاء وفهمها

استخدام الـ flagsالصح

معرفة مكان كل وظيفة فيPKGBUILD

==========

المرحلة — 11 نصائح الاحتراف النهائي

هدف المرحلة

تجعل أي PKGBUILD تكتبه مقروء، آمن، ونظيف

تعرف أفضل الممارسات قبل نشر باكدج على AUR أو توزيعتك

تتجنب الأخطاء التي يقع فيها معظم المبتدئين والمحترفين الجدد

11.1التعليقات واضحة = ذهب

كل مرحلة، كلdependency ، كل وظيفة :اكتب تعليق يشرح لماذا هي هنا

مثال:

```
#هذا السورس مطلوب لأنه البرنامج يحتاجhello.sh
source=('hello.sh')
sha256sums=('SKIP')
```

سبب :أي شخص يقرأ PKGBUILD بعد سنة يعرف كل شيء بدون تخمين

11.2استخدم prepare / build / check / package بذكاء

لا تخلط وظائفهم

كل stage في مكانها → يسهلdebug

أي مبتدئ يقدر يفهم PKGBUILD خلال دقيقة

11.3 split packages + Git sources

→ Gitاستخدم 'SKIP' + pkgver() للبصمة

→ split packagesاستخدم package_*() لكل باكدج منفصل

→ dependenciesكل باكدج يتحكم ب dependsالخاصة به

النتيجة PKGBUILD :نظيف،modular ، وصالح للنشر

= 11.4 dependenciesترتيب ذهني

runtime → depends

build time → makedepends

optional → optdepends

كل باكدج في split package يكتب dependencies الخاصة به

= 11.5 .installأداة، مش لعبة

استخدمها فقط لإشعارات أو تغييرات آمنة

لا تلمس النظام إلا إذا ضروري جدًا

رسائل توجيهية أفضل من تنفيذ commands تلقائيًا

11.6التسمية مهمة

باكدجات → Git ختمgit-

→ split packagesأسماء واضحة وموحدة

→ AUR & Archيحبوا وضوح الاسم والمستوى المهني

11.7اختبر باكدجك قبل النشر

makepkg -s → تثبيتdependencies

makepkg -f → إعادة البناء

شغّل check()

تأكد كل ملفات $pkgdir في مكانها

جرّب التثبيت والإزالة

خطوة أساسية قبل أي نشر في AUR أوHelwan Linux repo

11.8نصائح إضافية

اقرأPKGBUILD ات باكدجات مشهورة كمثال

احتفظ بPKGBUILD اتك(Git) version-controlled

اختبر باكدجاتك على Virtual Machine قبل النظام الرئيسي

لا تستخدم sudo فيPKGBUILD

11.9خلاصة المنهج

دلوقتي أنت:

تعرف كل متغير أساسي(pkgname, pkgver, pkgrel, source, sha256sums)

فاهم lifecycle كامل(prepare, build, check, package)

فاهم dependencies وأنواعها

تعرفsplit packages + Git sources

تعرف install. وكيفية استخدامها بأمان

تعرفconflicts, provides, replaces

تعرف debug وcommon mistakes

==========